Referência da API · Bibliotela

Contrato do servidor da Bibliotela, o app de livros, séries e filmes. Segue a estrutura da API da Carteira (mesmas rotas de login, perfil e erro, mesmos cabeçalhos de simulação) e troca os lançamentos por itens da biblioteca. Os exemplos são ilustrativos: o servidor ainda não existe, e este documento é a especificação que ele deve cumprir.

Visão geral

Verbo	Rota	Autenticação	Sucesso
`GET`	`/health`	não	200
`POST`	`/auth/login`	não	200
`POST`	`/auth/logout`	sim	204
`GET`	`/me`	sim	200
`PUT`	`/me`	sim	200
`GET`	`/genres`	sim	200
`GET`	`/items`	sim	200
`GET`	`/items/:id`	sim	200
`POST`	`/items`	sim	201
`PUT`	`/items/:id`	sim	200
`DELETE`	`/items/:id`	sim	204
`GET`	`/summary`	sim	200
`POST`	`/reset`	sim	204

Regras gerais

- **Base URL:** `http://localhost:8080`. No emulador Android, `http://10.0.2.2:8080`.
- **Formato:** corpo e resposta em JSON, com `Content-Type: application/json; charset=utf-8`.
- **Autenticação:** as rotas protegidas exigem o cabeçalho `Authorization: Bearer <token>`, com o token devolvido pelo login.
- **Dados por sessão:** cada login recebe uma cópia própria do seed. O que uma sessão altera, outra não vê.
- **Datas de leitura:** `startedAt` e `finishedAt` são só a data, `YYYY-MM-DD`, sem hora nem fuso. Elas dizem o dia em que a pessoa começou ou terminou, e não um instante: com hora em UTC, terminar um livro às 23h viraria o dia seguinte.
- **Data de criação:** `addedAt` é ISO 8601 em UTC (`2026-07-30T18:20:00Z`) e é gerada pelo servidor.
- **Nota:** inteiro de 1 a 10. O app mostra em estrelas dividindo por 2, com meia estrela (8 = 4 estrelas, 9 = 4,5). O servidor guarda só o inteiro.
- **Campos opcionais:** vêm sempre, com `null` quando vazios. Campos que não pertencem ao tipo do item também vêm `null`.
- **Latência:** de 300 a 800 ms por padrão (`LATENCY_MIN_MS`/`LATENCY_MAX_MS`).
- **CORS:** qualquer origem é aceita, e `OPTIONS` responde `204`.
- **Credenciais de teste:** `ana@bibliotela.dev` / `123456` e `bruno@bibliotela.dev` / `123456`.

Modelos

user

```
{
  "id": "usr_01",
  "name": "Ana Ribeiro",
  "email": "ana@bibliotela.dev",
  "avatarUrl": null,
  "createdAt": "2024-03-11T09:00:00Z"
}
```

item

```
{
  "id": "it_0042",
  "type": "book",
  "title": "Duna",
  "genre": "Ficção científica",
  "status": "done",
  "startedAt": "2026-08-12",
  "finishedAt": "2026-09-06",
  "rating": 9,
  "note": "Final inesperado",
  "pages": 680,
  "author": "Frank Herbert",
  "seasons": null,
  "durationMinutes": null,
  "addedAt": "2026-07-30T18:20:00Z"
}
```

Campo	Tipo	Regra
`type`	string	`book`, `series` ou `movie`
`title`	string	não vazio; espaços nas pontas são removidos
`genre`	string?	um dos valores de `GET /genres`; texto vazio vira null
`status`	string	`want`, `in_progress` ou `done` (ver tabela abaixo)
`startedAt`	string?	`YYYY-MM-DD`; regra por status, abaixo
`finishedAt`	string?	`YYYY-MM-DD`; regra por status, abaixo
`rating`	integer?	de 1 a 10; só vale com status `done`
`note`	string?	texto livre; texto vazio vira null
`pages`	integer?	só em `book`; maior que zero
`author`	string?	só em `book`; texto vazio vira null
`seasons`	integer?	só em `series`; maior que zero
`durationMinutes`	integer?	só em `movie`; maior que zero
`addedAt`	string	gerado pelo servidor; ignorado se vier no corpo

Regras por status:

status	startedAt	finishedAt	rating
`want`	null	null	null
`in_progress`	obrigatório	null	null
`done`	opcional	obrigatório	opcional

- `startedAt` não pode ser depois de `finishedAt`.
- Valores que não valem para o status ou para o tipo (uma `rating` num item `want`, `pages` num filme) são descartados e voltam `null` na resposta. Faltar o que é obrigatório dá 422.
- Os textos de status mudam por tipo só na tela: `want`/`in_progress`/`done` são “Quero ler / Lendo / Lido” para livro e “Quero assistir / Assistindo / Assistido” para filme e série.

`summary`

```
{
  "year": 2026,
  "books": { "finished": 12, "pages": 3480 },
  "movies": { "finished": 30, "minutes": 3650 },
  "series": { "finished": 5, "seasons": 11 },
  "averageRating": 7.8,
  "byGenre": [
    { "genre": "Ficção científica", "count": 14 },
    { "genre": "Drama", "count": 11 },
    { "genre": "Fantasia", "count": 8 },
    { "genre": "Suspense", "count": 6 },
    { "genre": null, "count": 4 },
    { "genre": "Comédia", "count": 4 }
  ]
}
```

Erro

Toda falha tem o mesmo formato. Em 422, o objeto ganha `fields`, com uma mensagem por campo inválido:

```
{
  "error": {
    "code": "validation",
    "message": "Invalid item",
    "fields": { "rating": "must be an integer from 1 to 10" }
  }
}
```

Status	code	Quando
400	`bad_request`	corpo não é JSON válido
401	`invalid_credentials`	e-mail ou senha errados no login
401	`unauthorized`	token ausente, desconhecido ou já encerrado
401	`token_expired`	só com `X-Fail: expired`
404	`not_found`	item ou rota inexistente
422	`validation`	campos ou parâmetros inválidos
500	`internal`	erro inesperado ou `X-Fail: 500`

Saúde

`GET /health`

Verifica se o servidor está no ar. É pública.

Requisição: sem corpo.

Resposta 200:

```
{ "status": "ok", "version": "0.1.0" }
```

Autenticação

`POST /auth/login`

Autentica o usuário e cria uma sessão com dados próprios. É pública.

Requisição:

```
POST /auth/login
Content-Type: application/json

{ "email": "ana@bibliotela.dev", "password": "123456" }
```

Resposta 200:

```
{
  "token": "tok_0vu34w77pn3oquql7xjtqp5r",
  "user": {
    "id": "usr_01",
    "name": "Ana Ribeiro",
    "email": "ana@bibliotela.dev",
    "avatarUrl": null,
    "createdAt": "2024-03-11T09:00:00Z"
  }
}
```

Erros:

Status	Exemplo
400	`{"error":{"code":"bad_request","message":"Malformed JSON body"}}`
401	`{"error":{"code":"invalid_credentials","message":"Invalid email or password"}}`
422	`fields: {"email":"must not be empty","password":"must not be empty"}`

`POST /auth/logout`

Encerra a sessão. Depois disso, o token passa a receber 401 `unauthorized`.

Requisição:

```
POST /auth/logout
Authorization: Bearer <token>
```

Resposta 204: sem corpo.

Erros: 401.

Perfil

`GET /me`

Devolve o usuário da sessão.

Requisição:

```
GET /me
Authorization: Bearer <token>
```

Resposta 200: um `user`.

Erros: 401.

`PUT /me`

Atualiza nome e avatar. Os outros campos do usuário não mudam.

Requisição:

```
PUT /me
Authorization: Bearer <token>
Content-Type: application/json

{ "name": "Ana R.", "avatarUrl": null }
```

Campo	Regra
`name`	obrigatório, não vazio; espaços nas pontas são removidos
`avatarUrl`	string ou null; texto vazio vira null

Resposta 200: o `user` atualizado.

Erros:

Status	Exemplo
400	`JSON malformado`
401	`token inválido`
422	`fields: {"name":"must not be empty"}`

Gêneros

`GET /genres`

Lista os gêneros aceitos em `genre`. O app usa esta lista para montar o formulário e as barras do resumo, sem deixar os nomes escritos no código.

Requisição:

```
GET /genres
Authorization: Bearer <token>
```

Resposta 200: ordem alfabética, com “Outros” no fim.

```
{
  "items": [
    "Ação", "Aventura", "Biografia", "Comédia", "Documentário", "Drama",
    "Fantasia", "Ficção científica", "Romance", "Suspense", "Terror", "Outros"
  ]
}
```

Erros: 401.

Itens

`GET /items`

Lista os itens da sessão, com filtros e paginação.

Requisição:

```
GET /items?year=2026&status=done&type=book&genre=Fantasia&q=duna&page=1&pageSize=20
Authorization: Bearer <token>
```

Parâmetro	Obrigatório	Padrão	Regra
`year`	não	todos	YYYY; filtra pela data de leitura (ver abaixo)
`status`	não	todos	`want`, `in_progress` ou `done`
`type`	não	todos	`book`, `series` ou `movie`

Parâmetro	Obrigatório	Padrão	Regra
`genre`	não	todos	um valor de `GET /genres`
`q`	não	—	busca em `title` e `author`, sem diferenciar maiúsculas
`page`	não	1	inteiro maior ou igual a 1
`pageSize`	não	20	inteiro de 1 a 100

Data de leitura. É `finishedAt` nos itens `done` e `startedAt` nos `in_progress`. É ela que define o ano do filtro `year`, a ordenação e o mês em que o app agrupa a lista. Itens `want` não têm data de leitura: **aparecem em qualquer ano** e o app os mostra num grupo “Na fila”.

Ordenação. Primeiro os itens com data de leitura, do mais recente para o mais antigo (empate: o mais novo em `addedAt`). Depois os `want`, do adicionado mais recentemente para o mais antigo. O agrupamento por mês fica contínuo, e os da fila ficam sempre no fim.

Resposta 200: uma página. `total` **conta os itens depois dos filtros.**


```
{
  "items": [
    {
      "id": "it_0060",
      "type": "movie",
      "title": "Oppenheimer",
      "genre": "Drama",
      "status": "done",
      "startedAt": "2026-09-27",
      "finishedAt": "2026-09-27",
      "rating": 8,
      "note": null,
      "pages": null,
      "author": null,
      "seasons": null,
      "durationMinutes": 181,
      "addedAt": "2026-09-10T21:05:00Z"
    },
    {
      "id": "it_0058",
      "type": "series",
      "title": "Severance",
      "genre": "Suspense",
      "status": "in_progress",
      "startedAt": "2026-09-20",
      "finishedAt": null,
      "rating": null,
      "note": "Segunda temporada",
      "pages": null,
      "author": null,
      "seasons": 2,
      "durationMinutes": null,
      "addedAt": "2026-09-18T12:40:00Z"
    }
  ],
  "page": 1,
  "pageSize": 2,
  "total": 18
}
```


Uma página além do fim devolve `items: []`, não um erro.

Erros:

Status	Exemplo
401	`token inválido`
422	`fields: {"year":"must be in the format YYYY","status":"must be one of: want, in_progress, done"}`

`GET /items/:id`

Devolve um item.

Requisição:

```
GET /items/it_0060
Authorization: Bearer <token>
```

Resposta 200: um `item`.

Erros:

Status	Exemplo
401	`token inválido`
404	`{"error":{"code":"not_found","message":"Item it_9999 not found"}}`

`POST /items`

Cria um item. O servidor gera o `id` e o `addedAt`; se vierem no corpo, são ignorados.

Requisição:

```
POST /items
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "book",
  "title": "Duna",
  "genre": "Ficção científica",
  "status": "done",
  "startedAt": "2026-08-12",
  "finishedAt": "2026-09-06",
  "rating": 9,
  "note": "Final inesperado",
  "pages": 680,
  "author": "Frank Herbert",
  "seasons": null,
  "durationMinutes": null
}
```

Resposta 201: o `item` criado, já normalizado.

```
{
  "id": "it_0081",
  "type": "book",
  "title": "Duna",
  "genre": "Ficção científica",
  "status": "done",
  "startedAt": "2026-08-12",
  "finishedAt": "2026-09-06",
  "rating": 9,
  "note": "Final inesperado",
  "pages": 680,
  "author": "Frank Herbert",
  "seasons": null,
  "durationMinutes": null,
  "addedAt": "2026-09-28T19:02:00Z"
}
```

Erros:

Status	Exemplo
400	`JSON malformado`
401	`token inválido`
422	`ver abaixo`

```
{
  "error": {
    "code": "validation",
    "message": "Invalid item",
    "fields": {
      "title": "must not be empty",
      "type": "must be one of: book, series, movie",
      "genre": "must be one of the values from /genres",
      "finishedAt": "is required when status is done",
      "startedAt": "is required when status is in_progress",
      "rating": "must be an integer from 1 to 10",
      "pages": "must be greater than zero"
    }
  }
}
```

Quando `startedAt` vem depois de `finishedAt`, o erro é `"startedAt": "must not be after finishedAt"`.

`PUT /items/:id`

Substitui um item inteiro. O corpo tem o mesmo formato do POST, e o `id` do caminho sempre prevalece sobre o do corpo. O `addedAt` original é mantido.

Requisição: terminando uma série que estava em andamento.

```
PUT /items/it_0058
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "series",
  "title": "Severance",
  "genre": "Suspense",
  "status": "done",
  "startedAt": "2026-09-20",
  "finishedAt": "2026-09-28",
  "rating": 9,
  "note": null,
  "pages": null,
  "author": null,
  "seasons": 2,
  "durationMinutes": null
}
```

Resposta 200: o `item` atualizado.

Erros: 400, 401, 422 (como no POST) e 404 quando o `id` não existe. A validação vem antes: um corpo inválido para um `id` inexistente recebe 422.

`DELETE /items/:id`

Remove um item.

Requisição:

```
DELETE /items/it_0058
Authorization: Bearer <token>
```

Resposta 204: sem corpo.

Erros: 401 e 404 (inclusive ao apagar duas vezes o mesmo `id`).

Resumo do ano

`GET /summary`

Soma o que foi concluído em um ano. Só entram itens `done` cujo `finishedAt` cai no ano pedido.

Requisição:

```
GET /summary?year=2026
Authorization: Bearer <token>
```

Parâmetro	Obrigatório	Padrão	Regra
`year`	não	ano atual	YYYY

- `books.finished`: livros concluídos no ano. `books.pages`: soma de `pages` desses livros (livro sem `pages` conta 0).
- `movies.finished` e `movies.minutes`: filmes concluídos e soma de `durationMinutes`.
- `series.finished` e `series.seasons`: séries concluídas e soma de `seasons`.
- `averageRating`: média das notas dos itens concluídos no ano que têm nota, com uma casa decimal; `null` se nenhum tem.
- `byGenre`: itens concluídos no ano agrupados por gênero, do maior para o menor. `genre: null` é “Sem gênero”.
- Um ano sem nada concluído devolve zeros, `averageRating: null` e `byGenre: []`.

Resposta 200:

```
{
  "year": 2026,
  "books": { "finished": 12, "pages": 3480 },
  "movies": { "finished": 30, "minutes": 3650 },
  "series": { "finished": 5, "seasons": 11 },
  "averageRating": 7.8,
  "byGenre": [
    { "genre": "Ficção científica", "count": 14 },
    { "genre": "Drama", "count": 11 },
    { "genre": null, "count": 4 }
  ]
}
```

Erros:

Status	Exemplo
401	`token inválido`
422	`fields: {"year":"must be in the format YYYY"}`

Utilitários

`POST /reset`

Devolve os dados da sessão ao estado do seed. O token continua válido.

Requisição:

```
POST /reset
Authorization: Bearer <token>
```

Resposta 204: sem corpo.

Erros: 401.

Cabeçalhos de simulação

Valem em qualquer rota e são avaliados antes da autenticação.

Cabeçalho	Efeito
`X-Delay-Ms: 3000`	`usa exatamente esta latência, em ms`
`X-Fail: 500`	`500 com {"error":{"code":"internal","message":"Simulated failure"}}`
`X-Fail: expired`	`401 com {"error":{"code":"token_expired","message":"Token has expired"}}`
`X-Fail: timeout`	`segura a requisição por 60 s e responde 504 em texto puro`

Com um token válido, uma rota que não existe responde 404 com `{"error":{"code":"not_found","message":"Route <caminho> not found"}}`.

Seed

- **Usuários** (senha `123456` para todos): `usr_01` Ana Ribeiro (`ana@bibliotela.dev`) e `usr_02` Bruno Costa (`bruno@bibliotela.dev`).
- **Volume e anos:** cerca de 45 itens para Ana e 15 para Bruno, concluídos em 2025 e em 2026, para o filtro `year` e o resumo terem o que mostrar.
- **Variedade de propósito:** os três tipos, os três status, itens com e sem `genre`, `note`, `rating` e `author`, livros com e sem `pages`, e itens `want` sem nenhuma data.